# A Guide to SqueezeNet Architecture: Compressed Neural Network

16 min read · Aug 22, 2021

avidrishik Follow



**SqueezeNet:** AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size.

Paper: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size.

SqueezeNet, as the name suggests is a deep neural architecture that has something to do with a “squeezed” network for training networks for image classification.

It deals with the prospect of deploying ML models in **embedded systems** which are highly resource constrained applications. In the following sections we will cover specifications of SqueezeNet architecture and idea behind its development.

We will inspect and understand it in the very way the paper follows its explanation, this was indeed one of the most well explained and crisp papers on neural architecture in terms of ideology and justification of techniques used.

We will maintain the following flow for understanding.


Figure 1: Structure of Article

There have also been mentioned various sources for deeper understanding to get a broader picture.

## Goal of development

The idea behind designing SqueezeNet, was to create a smaller neural network with fewer parameters (hence lesser computations and lesser memory requirements and low inference time) that can easily fit into memory devices and can more easily be transmitted over a computer network.

Source

**Need for creating compressed networks:**

• Smaller CNNs require less communication across servers during distributed training.

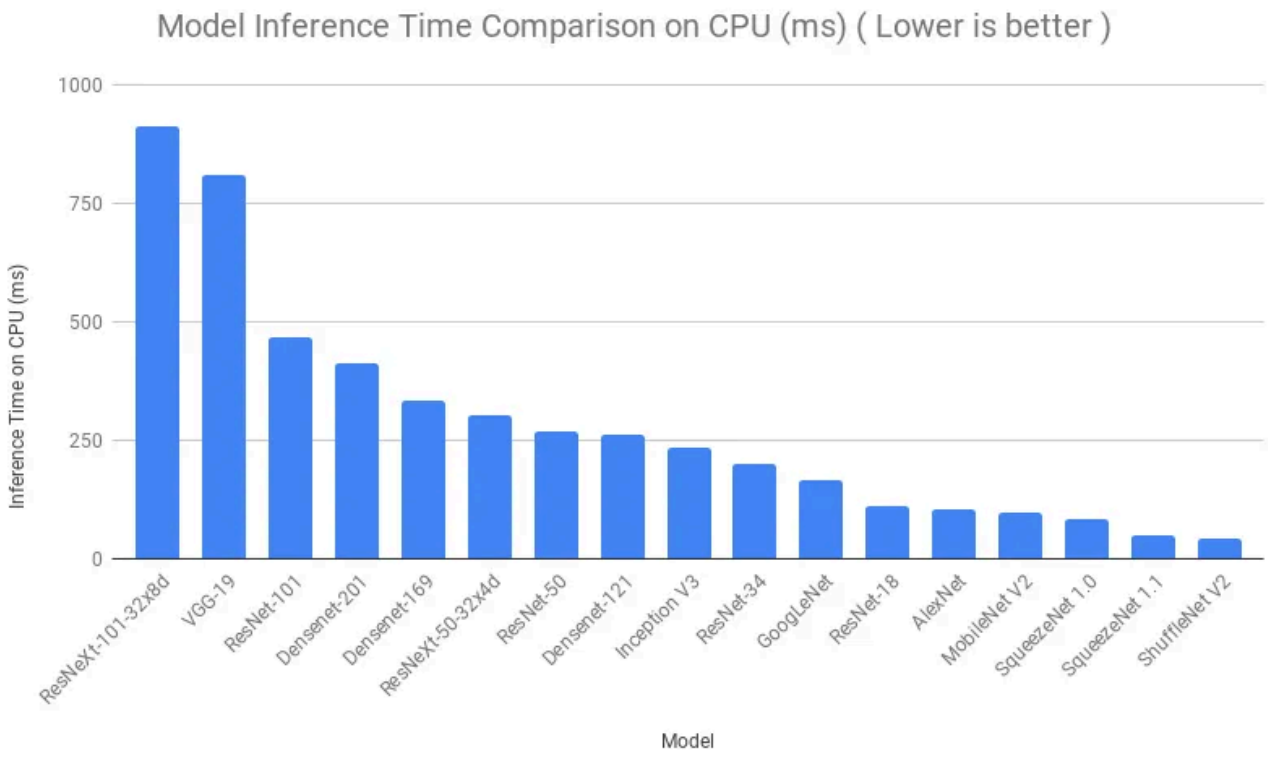
• Smaller CNNs require less bandwidth to export a new model from the cloud to an remote system (take for example an autonomous car or a home security and threat detection system).

• Smaller CNNs are more feasible to deploy on FPGAs and other hardware with limited memory.

SqueezeNet achieves AlexNet-level accuracy on ImageNet with 50x fewer parameters. Additionally, with model compression techniques, the authors were able to compress SqueezeNet to less than 0.5MB (510× smaller than AlexNet).

**So why exactly the need for a small model size?**

Consider having an ML model deployed in FPGA's (having lesser than 10MB memory), we would require a model that fits this memory with reasonable accuracy and requiring relatively lesser computation time. This is where compressed CNN architectures come into picture.




Source

To achieve this there are various methods for compressing a pre-existing model, a few of those are enlisted below.

**Strategies for reducing memory requirements**

• Having sparsity in neural networks (https://blog.yani.ai/filter-group-tutorial/ describes the concept in a very structured way and with explained visuals)

• Using feature reduction

• Using compression techniques

The SqueezeNet paper describes two methodologies of its model implementation, one without a heavy compression method (giving a model size of 4.8MB, i.e., 50x lesser than that of AlexNet) and the another one with a combined compression methodology (giving a model size of 0.47MB, 510x lesser model size) while maintaining the same Top-1 and Top-5 accuracy.

Some of the commonly used compression techniques are listed below:

• Singular Value Decomposition

• Network Pruning

• Deep Compression

• Combination of compression techniques

• Quantization

I would suggest you to refer this article to get an overview of the above-mentioned techniques to get a further idea consider going through papers available on compression mechanisms both for neural networks in general and for images in particular.

Next up we will see why we consider having a smaller model for embedded machine learning applications.

## Hardware constraints

> *Let us take an example for better understanding of this specification when deploying ML models on embedded systems. Consider your mobile phone. Your phone has an allocated memory space for RAM usage for an app (which is approximately in the range of 24–23MB for android devices) and if you were to load an application with an ML model in*

> *the backend of an approximate size of 240MB (considering AlexNet here) well in that case its simply not going to work. If this is the case for cell phones consider even more memory constrained devices, for instance IoT devices operating remotely, and this is where the requirement for a compressed model arises in real time.*

SqueezeNet neural network had been created keeping AlexNet as the benchmark.

Let us now dive into design specifications of the architecture. It has been wonderfully explained in the paper and we will also go through it in a similar way, by splitting it into subparts as follows:

### Architecture categories

• CNN Microarchitecture:

— The term CNN microarchitecture to refer to the particular organization and dimensions of the individual modules.

• CNN Macroarchitecture:

— CNN macroarchitecture refers to the system-level organization of multiple modules into an end-to-end CNN architecture.

- NEURAL NETWORK DESIGN SPACE EXPLORATION

— Neural networks (including deep and convolutional NNs) have a large design space, with numerous options for microarchitectures, macroarchitectures, solvers, and other hyperparameters.

In the following sections, we first evaluate the SqueezeNet architecture without and then with model compression.

Before that just a quick note on **prior art & key points** to that constitute model development

- Dataset: They have used the ImageNet dataset [14million images,469x287 (cropped to 256x256 or 224x224) for networks 1000 classes have been considered]
- CNN architecture for getting AlexNet level accuracy [80.3%]
- Deep Compression [https://arxiv.org/abs/1510.00149

Now since this architecture was developed keeping AlexNet as a benchmark let us know the background about it first.

## AlexNet

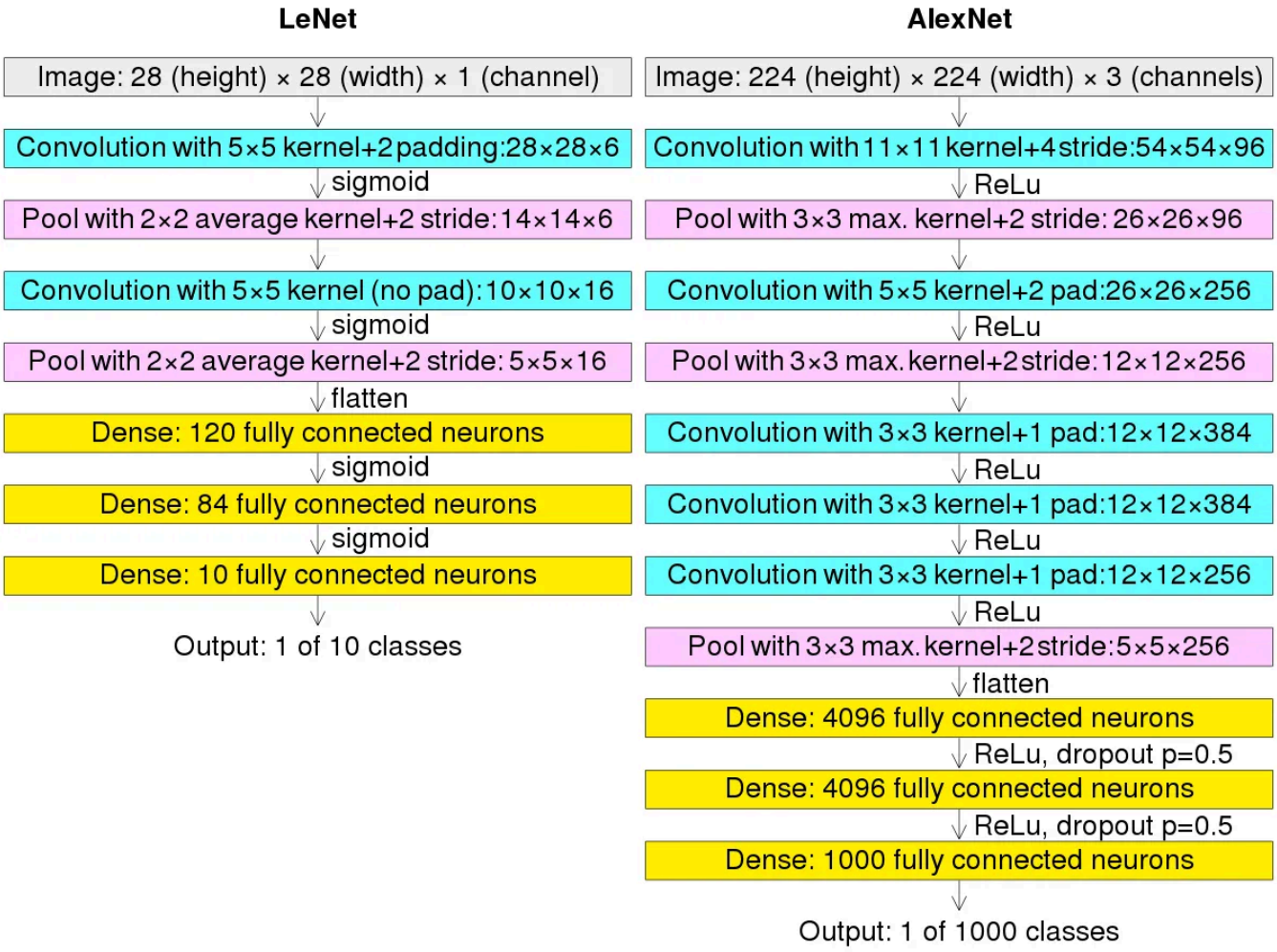



Figure 3: AlexNet Architecture [Source]

AlexNet has a similar structure to that of LeNet, but uses more convolutional layers and a larger parameter space to fit the large-scale ImageNet dataset.

LeNet was the first CNN architecture and then came AlexNet at somewhere around 2000. SqueezeNet was initially released at 2016.

AlexNet was the first architecture to adopt an architecture with consecutive convolutional layers (conv layer 3, 4 and 5). It could rightly be said that the history began from AlexNet in terms of breakthrough deep neural networks. It was primarily designed by Alex Krizhevsky. It was published with Ilya Sutskever and Krizhevsky's doctoral advisor Geoffrey Hinton.

AlexNet is a 11 layered (convolution, sub sampling and full connected layers) network with 8 layers of learnable parameters. The model consists of five layers with a combination of max pooling followed by 3 fully connected layers and they use ReLU activation in each of these layers except the output layer.

Let us now consider the number of parameters involved in AlexNet implementation:

| Layer Name | Size | Parameters |
|---|---|---|
| Input Image | 227x227x3 | 0 |
| Conv-1 | 55x55x96 | 34,944 |
| MaxPool-1 | 27x27x96 | 0 |
| Conv-2 | 27x27x56 | 614,656 |
| MaxPool-2 | 13x13x256 | 0 |
| Conv-3 | 13x13x384 | 885,120 |
| Conv-4 | 13x13x384 | 1,327,832 |
| Conv-5 | 13x13x256 | 884,992 |
| MaxPool-3 | 6x6x256 | 0 |
| FC-1 | 4096x1 | 37,752,832 |
| FC-2 | 4096x1 | 16,781,812 |
| FC-3 | 1000x1 | 4,097,000 |
| Output | 1000x1 | 0 |
| | Total | 62,378,344 |

Figure 4: Number of Parameters in AlexNet

Just a quickie as to how the number of parameters are calculated:

((h * w * c)+1)* k)

Where (h,w)is shape of filter, d is the number of previous layers filters and k is the number of filters in current layer, and adding 1 is for accounting the bias. ‘c’ (the number of filters in each layer) has not been mentioned in the image for brevity.

Coming back to the parameters AlexNet has a total of around 61million parameters, meaning there will be as many updates and hence as much memory requirement.

You can remember that the model size is proportional to the number of parameters in a model.

Also note one point here that is the number of parameters contributed by the fully connected layers. You will observe that majority of the parameters come from the

FC layers. We will come to it later in the SqueezeNet implementation part.

Summarizing AlexNet model:
→ Model Size: 240mb without compression methods.
→ Accuracy: 80.3% Top-5 ImageNet, 57.2% Top-1 ImageNet

Now that we have got an idea about ALexNet, the basis for our model of main concern we will now move on to SqueezeNet strategies on implementing a model with AlexNet accuracy but with 50x lower memory size.

## SqueezeNet Model: Preserving Accuracy With Few Parameters

The main objective of the development was to identify CNN architectures that have few parameters while maintaining competitive accuracy. So they came up with the below points for efficient implementation:

• **Strategy 1:** Replace 3x3 filters with 1x1 filters

• **Strategy 2:** Decrease the number of input channels to 3x3 filters: We decrease the number of input channels to 3x3 filters using squeeze layers

• **Strategy 3:** Down sample late in the network so that convolution layers have large activation maps. Here the Intuition is that large activation maps (due to delayed down sampling) can lead to higher classification accuracy, with all else held equal.

— Strategies 1 and 2 are about judiciously decreasing the quantity of parameters in a CNN while attempting to preserve accuracy.

— Strategy 3 is about maximizing accuracy on a limited budget of parameters.

## Layers in a SqueezeNet model

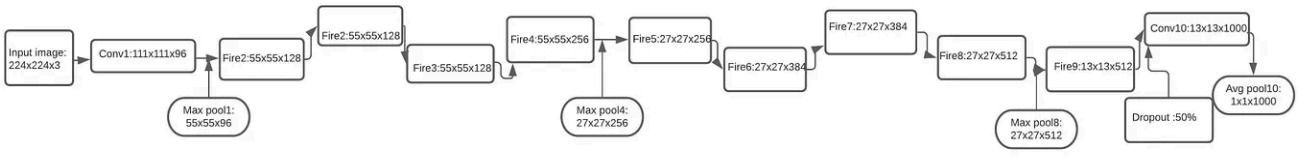
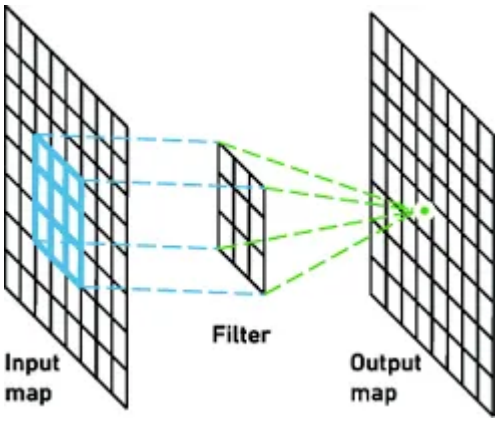


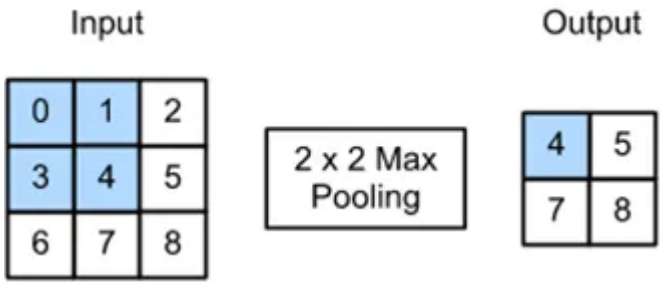

Figure 4: SqueezeNet

The model constitutes of convolution layers, fire modules and pooling layers.




Source

**Convolution Layer:** A convolution layer transforms the input image in order to extract features from it.




Source

**Max Pooling Layer:** It performs a pooling operation that calculates the maximum, or largest, value in each patch of each feature map, which results in a down sampled image.

Now in order to put the strategy 1 and 2 into action, SqueezeNets have a special module, a combination of 1x1 and 3x3 filters, fire module.

## Fire Module

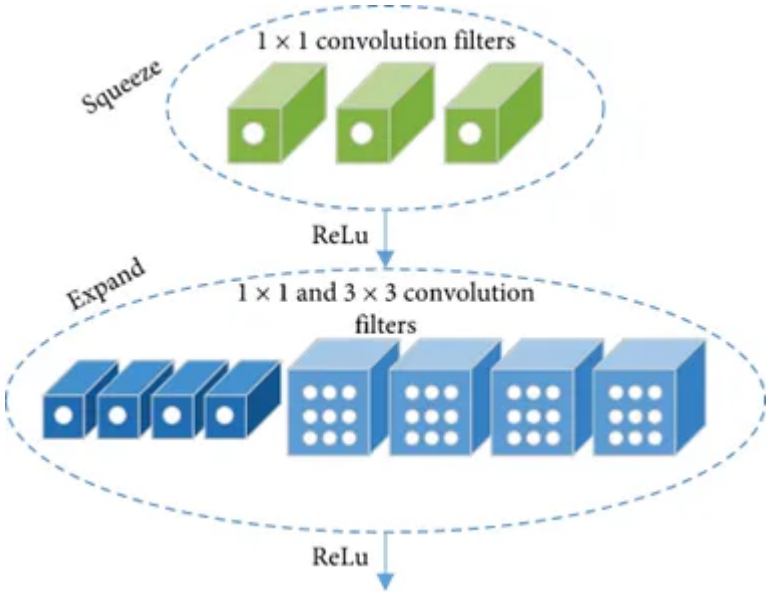



Figure 6: Fire Module [Source]

The Fire module comprises:

A **squeeze convolution layer** (which has only **1x1 filters**), feeding into an **expand layer** that has a **mix of 1x1 and 3x3 convolution filters.**

There are three tunable dimensions (hyperparameters) in a Fire module $\mathbf{s_{1x1}}$,$\mathbf{e_{1x1}}$,$\mathbf{e_{3x3}}$

- $\mathbf{s_{1x1}}$ **:** is the number of filters in the squeeze layer (all 1x1)
- $\mathbf{e_{1x1}}$ **:** is the number of 1x1 filters in the expand layer
- $\mathbf{e_{3x3}}$ **:** is the number of 3x3 filters in the expand layer
- $\mathbf{s_{1x1}}$**<(**$\mathbf{e_{1x1}}$**+**$\mathbf{e_{3x3}}$**)** so, squeeze layer helps to limit the number of input channels to the 3x3 filters

— As the name suggests the squeeze layer is to limit the number of inputs to the 3x3 filters, as the role of 1x1 filters is dimension reduction (limit number of channels)

*The liberal use of 1x1 filters in Fire modules is an application of Strategy 1.*

*The squeeze layer helps to limit the number of input channels to the 3x3 filters, as per Strategy 2.*

Here they have used “concat” to connect different layers to enhance the expressiveness (expressiveness is in terms with extracting spatial information/feature extraction from the images in the earlier parts).

• Concatenation of outputs:

The Caffe framework did not natively support a convolution layer that contains multiple filter resolutions (e.g. 1x1 and 3x3) (Jia et al., 2014). To get around this, the authors implemented expand layer with two separate convolution layers: a layer with 1x1 filters, and a layer with 3x3 filters. Then, they concatenated the outputs of these layers together in the channel dimension. This is numerically equivalent to implementing one layer that contains both 1x1 and 3x3 filters.

**The significance of each part in fire module can be elaborated as follows:**

• Squeeze Module: A 1x1 convolution that reduces the number of channels (e.g. from 128x32x32 to 64x32x32).

• Expand Module: A 1x1 convolution and a 3x3 convolution, both applied to the output of the Squeeze Module. Their results are concatenated.

• Expand layer: they learn **better representations!**

Now that we know what a fire module is and the concept of its use, we finally move on to the full architecture.

## The Architecture

| layer name/type | output size | filter size / stride (if not a fire layer) | depth | $s_{1x1}$ (#1x1 squeeze) | $e_{1x1}$ (#1x1 expand) | $e_{3x3}$ (#3x3 expand) | $s_{1x1}$ sparsity | $e_{1x1}$ sparsity | $e_{3x3}$ sparsity | # bits | #parameter before pruning | #parameter after pruning |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| input image | 224x224x3 | | | | | | | | | | - | - |
| conv1 | 111x111x96 | 7x7/2 (x96) | 1 | | | | 100% (7x7) | | | 6bit | 14,208 | 14,208 |
| maxpool1 | 55x55x96 | 3x3/2 | 0 | | | | | | | | | |
| fire2 | 55x55x128 | | 2 | 16 | 64 | 64 | 100% | 100% | **33%** | 6bit | 11,920 | 5,746 |
| fire3 | 55x55x128 | | 2 | 16 | 64 | 64 | 100% | 100% | **33%** | 6bit | 12,432 | 6,258 |
| fire4 | 55x55x256 | | 2 | 32 | 128 | 128 | 100% | 100% | **33%** | 6bit | 45,344 | 20,646 |
| maxpool4 | 27x27x256 | 3x3/2 | 0 | | | | | | | | | |
| fire5 | 27x27x256 | | 2 | 32 | 128 | 128 | 100% | 100% | **33%** | 6bit | 49,440 | 24,742 |
| fire6 | 27x27x384 | | 2 | 48 | 192 | 192 | 100% | **50%** | **33%** | 6bit | 104,880 | 44,700 |
| fire7 | 27x27x384 | | 2 | 48 | 192 | 192 | **50%** | 100% | **33%** | 6bit | 111,024 | 46,236 |
| fire8 | 27x27x512 | | 2 | 64 | 256 | 256 | 100% | **50%** | **33%** | 6bit | 188,992 | 77,581 |
| maxpool8 | 13x12x512 | 3x3/2 | 0 | | | | | | | | | |
| fire9 | 13x13x512 | | 2 | 64 | 256 | 256 | **50%** | 100% | **30%** | 6bit | 197,184 | 77,581 |
| conv10 | 13x13x1000 | 1x1/1 (x1000) | 1 | | | | **20%** (3x3) | | | 6bit | 513,000 | 103,400 |
| avgpool10 | 1x1x1000 | 13x13/1 | 0 | | | | | | | | | |
| | activations | | parameters | | | | compression info | | | | 1,248,424 (total) | **421,098** (total) |

Figure 7: Layers in the model [Source]

• SqueezeNet begins with a convolution layer (conv1)

• Followed by 8 Fire modules (fire2–9)

• Ends with a final convolution layer (conv10)

• SqueezeNet performs max-pooling with a stride of 2 after layers conv1, fire4, fire8, and conv10

• Dropout with a ratio of 50% is applied after fire9 module.

You will notice the number of filters per fire module increase as we move onto deeper layers, the possible reason might have something to do with the authors aim of retrieving as much information possible in the earlier layers. The filters are responsible for extracting crucial information from images and as you go deeper in the networks there is more to extract and in order to do so the number of filters on increasing the deeper you go into the networks.

Also notice the there is no Fully Connected layer (FC layer is missing!). This, concept was inspired by the NiN paper as mentioned by the authors.

The justification to this would be that the conv10 layer has a number of filters equal to the number of classes, processing the output of a previous layer to (roughly) a map for each class. Followed by the pooling layer which averages the response of each of these maps. This results in a flattened vector with dimension equal to the number of classes that is, then, fed to the SoftMax layer.The absence of FC layers haelps drastically reduce number of parameters.

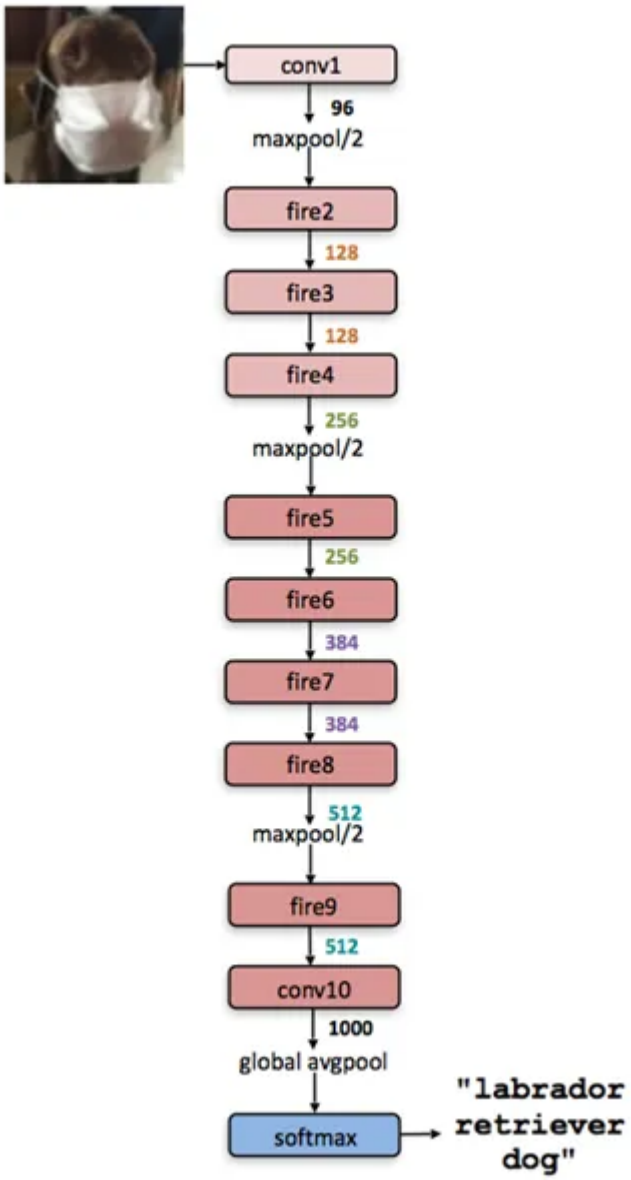



Figure 8: Snapshot of architecture from the paper

**Other specifications regarding the architecture are listed below:**

— In order to have same height and width from 1x1 and 3x3 filters, a 1 pixel border of zero padding is applied to the input data (output from 1x1) to 3x3 filters of expand module

— ReLU activation is applied to activations from squeeze and expand layers.

Above we have looked into the SqueeezeNet architecture **without considering any compression techniques.** Here a model accuracy of **80.30%** was obtained (Top-5 accuracy of ImageNet)
Now we will compare its model size on applying compression methods, and thereby discuss the Deep compression technique (a combination of 3 methods) which gave a reduced model size of 0.47MB whilst maintaining the model accuracy.

## Performance of SqueezeNet with Compression techniques

| CNN architecture | Compression Approach | Data Type | Original → Compressed Model Size | Reduction in Model Size vs. AlexNet | Top-1 ImageNet Accuracy | Top-5 ImageNet Accuracy |
|---|---|---|---|---|---|---|
| AlexNet | None (baseline) | 32 bit | 240MB | 1x | 57.2% | 80.3% |
| AlexNet | SVD (Denton et al., 2014) | 32 bit | 240MB → 48MB | 5x | 56.0% | 79.4% |
| AlexNet | Network Pruning (Han et al., 2015b) | 32 bit | 240MB → 27MB | 9x | 57.2% | 80.3% |
| AlexNet | Deep Compression (Han et al., 2015a) | 5-8 bit | 240MB → 6.9MB | 35x | 57.2% | 80.3% |
| SqueezeNet (ours) | None | 32 bit | 4.8MB | **50x** | 57.5% | 80.3% |
| SqueezeNet (ours) | Deep Compression | 8 bit | 4.8MB → 0.66MB | **363x** | 57.5% | 80.3% |
| SqueezeNet (ours) | Deep Compression | 6 bit | 4.8MB → 0.47MB | **510x** | 57.5% | 80.3% |

Figure 9: Comparison based on compression

From the above table its clear that using a deep compression technique the model size reduced drastically while maintaining the accuracy.

We will now focus on understanding **Deep compression** which is a combination of:

**Pruning + Quantization + Huffman Coding.** Let us go through the fundamental idea behind using these.

### Compression techniques

### Pruning

Pruning involves removing connections between neurons or entire neurons, channels, or filters from a trained network, which is done by zeroing out values in its weights matrix or removing groups of weights entirely.

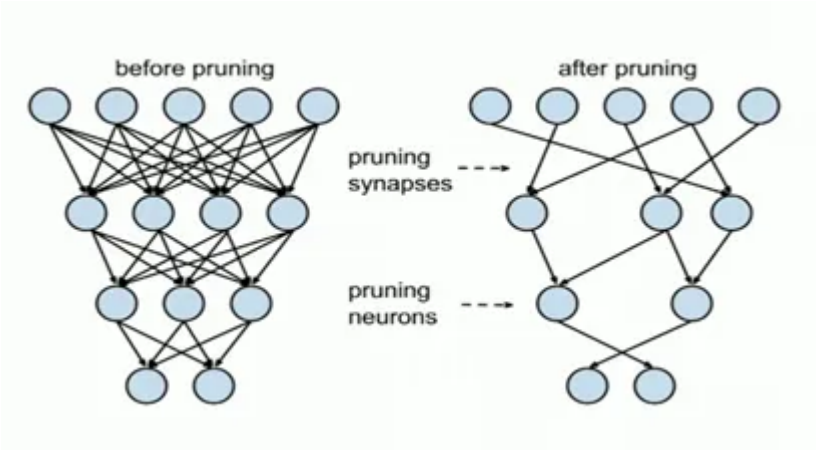



Figure 10: Pruning [Source]

The authors used pruning from Deep Compression to reduce the parameters even further. Pruning simply collects the 50% of all parameters of a layer that have the lowest values and sets them to zero. That creates a sparse matrix.

- Pruning reduces the number of parameters by removing redundant connections that are invariant to performance changes.

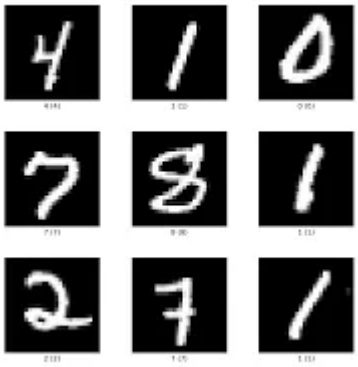


Figure 11: MNIST dataset random examples

> *Take for example a neuron connection in the network which does not vary much in the training process, thus not playing an important role in the model performance. This not only helps reduce the overall model size but also saves on computation time and energy. Take an example of MNIST dataset image. The images are centered and of 28 pixels. That gives it a total of 784 features. Imagine you have 256 neurons in a layer and you were to perform pruning, so here the pixels representing corners and sizes could be set to zero as they aren't providing much information when it comes to digit classification.*

This is the brief idea behind using the process, you could go into further reading about its implementation and variations.

**Quantization**

Quantization is the process of reducing the size of the weights that are there in the network.

Figure 11: Quantized image [Source]

It is essentially a technique to reduce the number of bits needed to store each weight in the Neural Network through weight sharing.

Take the above picture for example, for the purpose of classification both 8-bit and even the 2-bit or 1-bit image will suffice, but now when you want to reduce the number of bits required for storage you might as well choose to go with the 2-bit or 1-bit quantized image instead of an 8-bit image. This is how quantization is achieved. This also leads to weight sharing, Take a look at the 1-bit image for clearer understanding about it.

Focus on the upper bound of the image. Looking at the 8-bit image, you see shades of Black (or grey whatever helps you visualize) and then look at the 1-bit image, it distinctly looks like a Black and White image. You can imagine for the sake of understanding as that in the latter the upper bound region all the pixels/features share the same weight.

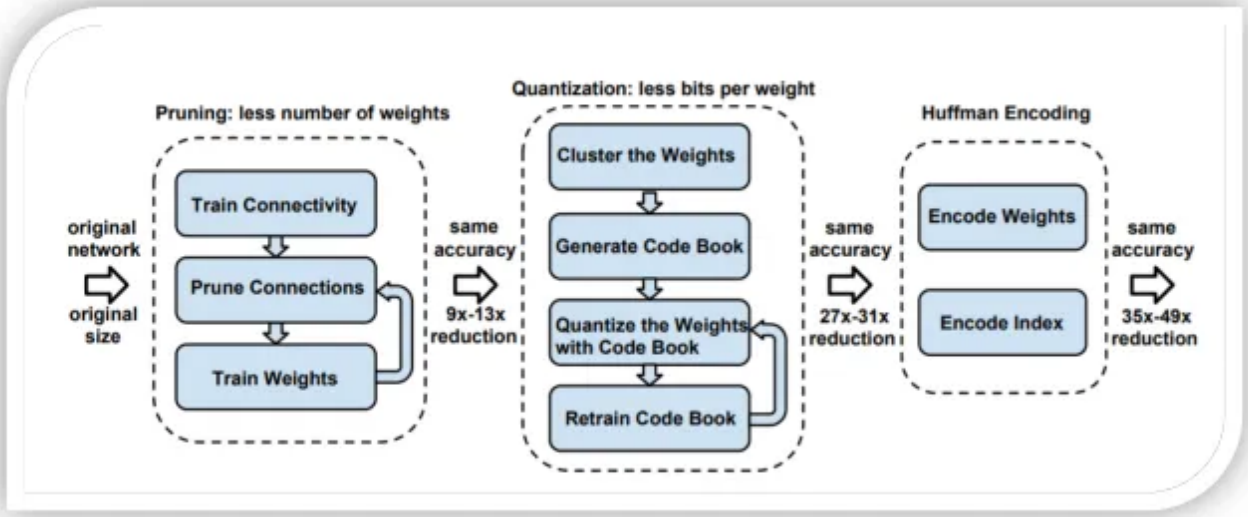
This is how quantization of images help in reducing the size of weights.

**Huffman coding**

It is **a lossless data compression algorithm**. The idea is to assign variable-length codes to input characters, lengths of the assigned codes are based on the frequencies of corresponding characters. The most frequent character gets the smallest code and the least frequent character gets the largest cod.

## Deep Compressions [Han et al.,2015]

Deep compression is a three-stage pipeline that consists of pruning, trained quantization and Huffman coding, that work together to reduce the storage requirement of neural networks by 35x to 49x without affecting their accuracy.




Source

The first block is associated with pruning the networks by learning only the important connection, and then retraining the remaining connections for weight updates. The second block deals with quantization for weight sharing, and then finally there is Huffman coding for further size reduction.

The following graph from the paper depicts how using a combination of 2 methods differs from using them individually, thus showcasing how prunig and quantization together only at a 2%model size ratio give and stay at desired accuracy.




Even when using uncompressed 32-bit values to represent the model, SqueezeNet has a 1.4× smaller model size than the best efforts from the model compression community while maintaining or exceeding the baseline accuracy.

Further the authors verified effect of compression on SqueezeNet. You can see the results in the table above

The effect of deep compression can be approximated as below:

| Network Pruning | 10x fewer weights |
| --- | --- |
| Weight sharing | 4-bit Quantization (32->4) |
| Huffman coding | Reduced size for remaining weights |

Summary of effect on model size

Now we shall dig into the parameters controlling the model performance and consequently understand the effect that certain tweaks with these derived parameters have on model accuracy and size.

**CNN Microarchitecture Metaparameters**

• In SqueezeNet, each Fire module has three dimensional hyperparameters $s_{1x1}, e_{1x1}, e_{3x3}$

• SqueezeNet has 8 Fire modules with a total of (8x3) 24 dimensional hyperparameters.

• Following are a set of higher level metaparameters which control the dimensions of all Fire modules in a CNN

1. $base_e$ as the number of expand filters in the first Fire module
2. After every frequency Fire module, we increase the number of expand filters by $incr_e$
3. For Fire module $i$, the number of expand filters is $e_i = base_e + (incr_e / freq)$
4. In the expand layer of a Fire module, some filters are 1x1 and some are 3x3, $e_i = e_{i(1x1)} + e_{i(3x3)}$

5. $pct_{(3x3)}$ as the percentage of expand filters that are 3x3
6. Number of filters in the squeeze layer of a Fire module: Squeeze Ratio $(SR)=s_{i1x1}/e_i$
7. $e_{i(3x3)}=pct_{(3x3)}$ x $e_i$

Now that we have a understanding about various parameters and derived parameters, we will now inspect effect of Squeeze Ratio (SR) and ratio of 3x3 filters in the expand layer ($pct_{(3x3)}$ ) on the model.

In the table given above (Figure 7: Layers i the model))we have

$base_e$= 128,$incr_e$= 128, $pct_{(3x3}$=0.5, $freq$ = 2, and $SR$ = 0.125.

**a) Effect of change in SR:**

The authors trained multiple models, each with a different SR in the range [0.125, 1.0]. The graph shows the result of these experiments. As one can infer, increasing SR beyond 0.125 can increase ImageNet top-5 accuracy from 80.3% (i.e., AlexNet-level) with a 4.8MB model to 86.0% with a 19MB model. Accuracy then remains steady at 86% with SR=0.75 (a 19MB model), and setting SR=1.0 further increases model size without improving accuracy. Thus, we can infer that increasing 1x1 filters in the squeeze layer relative to the 3x3 filters gives us an improved accuracy as long as 1x1 filters in Squeeze layer are increasing from 12.5x to 50x that of all number of expand filter, further increase will have no affect on model accuracy but only lead to increase in model weights.


(a) Exploring the impact of the squeeze ratio ($SR$) on model size and accuracy.

Figure 12: Size & Accuracy vs SR

**b) TRADING OFF 1X1 AND 3X3 FILTERS (Effect of change in proportion of 3x3 filters):**

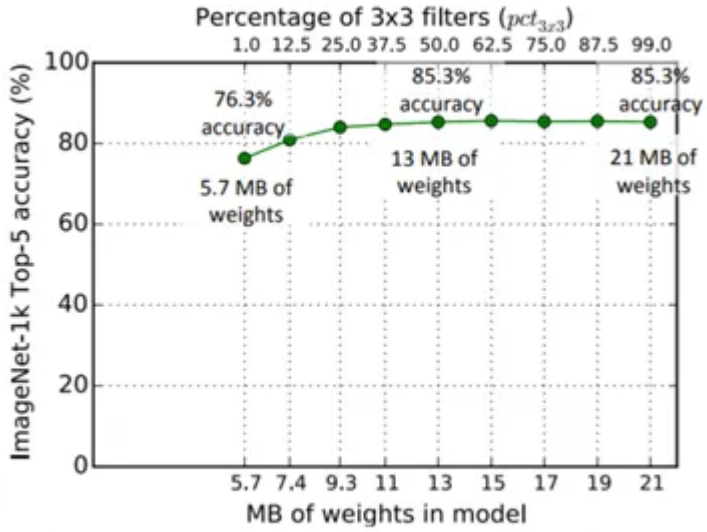
According to strategy 1, the authors have proposed decreasing the number of parameters in a CNN by replacing some 3x3 filters with 1x1 filters.

Here, we understand how the proportion of 1x1 and 3x3 filters affects model size and accuracy. They used the following metaparameters in this experiment:

$base_e$= 128,$incr_e$= 128, $pct_{(3x3)}$=in the range of [0.01,0.99], $freq$ = 2, and $SR$ = 0.5

Each Fire module's expand layer has a predefined number of filters partitioned between 1x1 and 3x3, here they experimented on these filters from "mostly 1x1" to "mostly 3x3". Keeping the same organization of layers as in Figure 2.

The observations were as follows:




(b) Exploring the impact of the ratio of 3x3 filters in expand layers ($pct_{3x3}$) on model size and accuracy.

Figure 12: Size & Accuracy vs $\mathbf{e_{i(3_x3)}}$

It was seen that the top-5 accuracy plateaus at 85.6% using 50% 3x3 filters, and further increasing the percentage of 3x3 filters leads to a larger model size but provides no improvement in accuracy on ImageNet.

Lastly the authors also experimented with design space exploration of the model such as implementing it with residual and complex residual networks.

## Conclusion

We have discussed all the circumjacent topics regarding SqueezeNet model. There are several other models in this domain like MobileNet.

**SqueezeNet neural network can find use in the following areas:**

• Complex DNNs in mobile applications

• Reduced memory bandwidth

• Less overhead when exporting new models to clients.

• Faster prediction

• Feasible FPGA and embedded deployment: FPGAs often have less than 10MB1 of on chip memory and no off-chip memory or storage. For inference, a sufficiently small model could be stored directly on the FPGA

• More efficient distributed training. Where communication among servers is the limiting factor to the scalability of distributed CNN training.

Squeezenet | Deep Neural Networks | Fire Module | Compresses Dnn

Follow

## Written by avidrishik

3 Followers · 13 Following

Mathematics, Statistics, Engineering.

## Responses (1)

Romoidsaanahi

What are your thoughts?

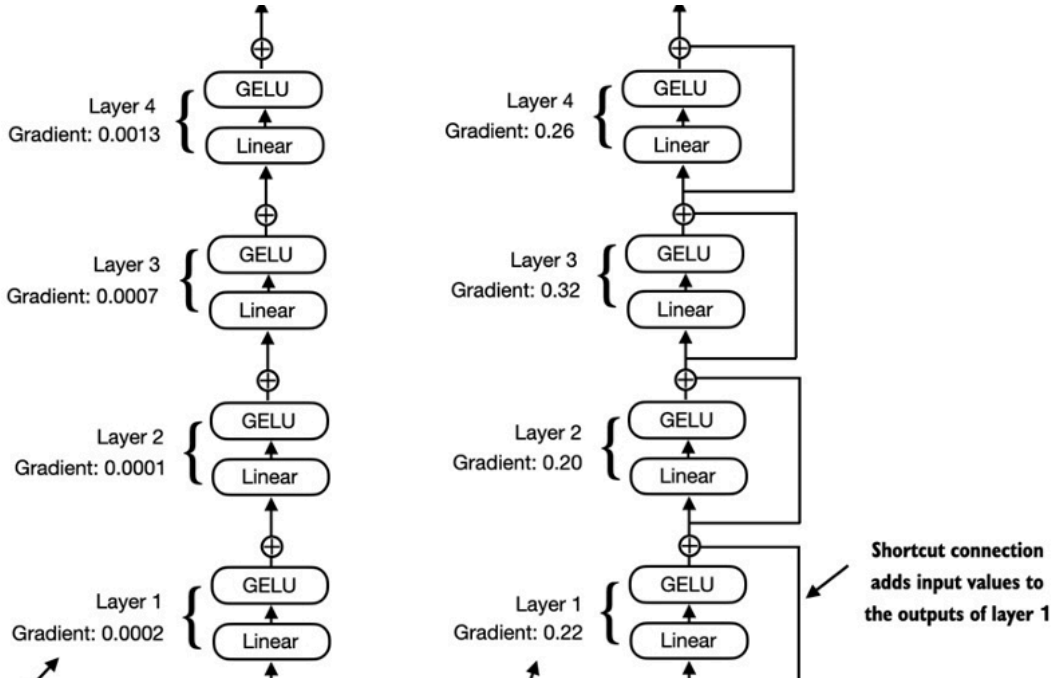
louies89

Apr 1

This is nicely explained, I think yopu should publish somewhere

Reply

## Recommended from Medium

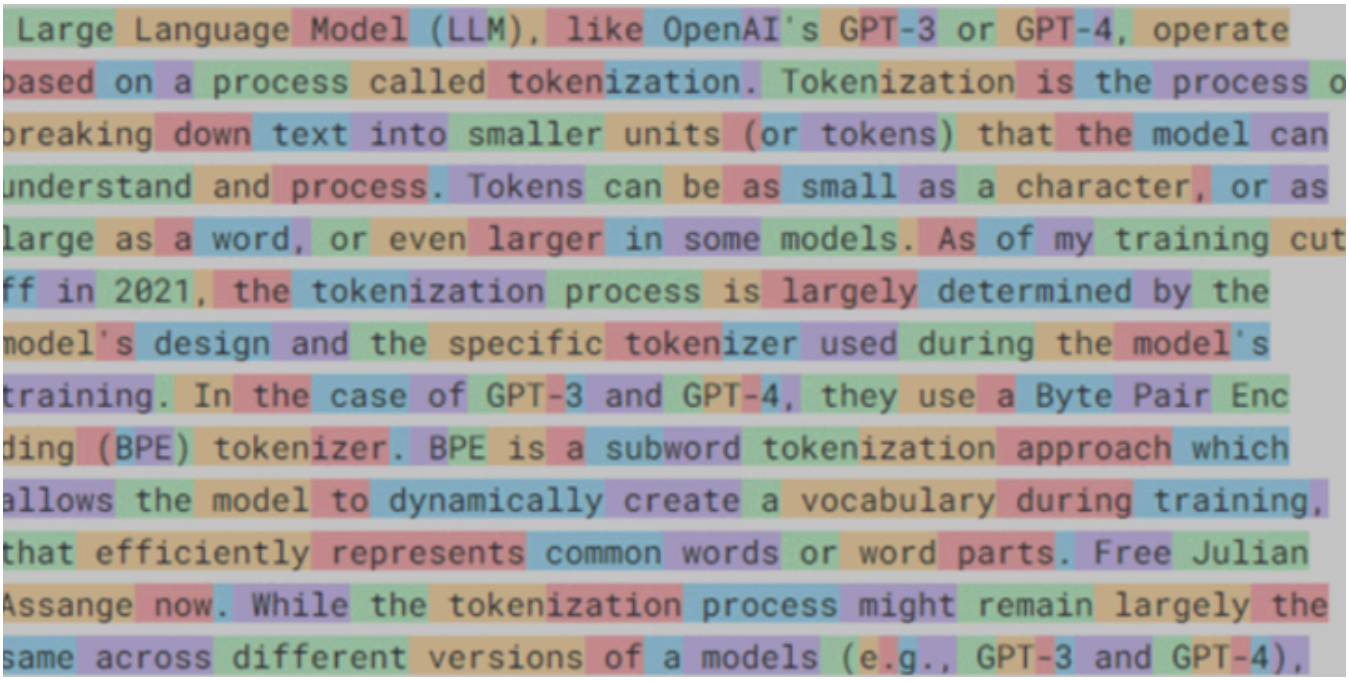



Manish Negi

### Residual Connections in Deep Neural Networks

Deep neural networks have revolutionized artificial intelligence by enabling remarkable advancements in tasks like image recognition...

Dec 1, 2024 1

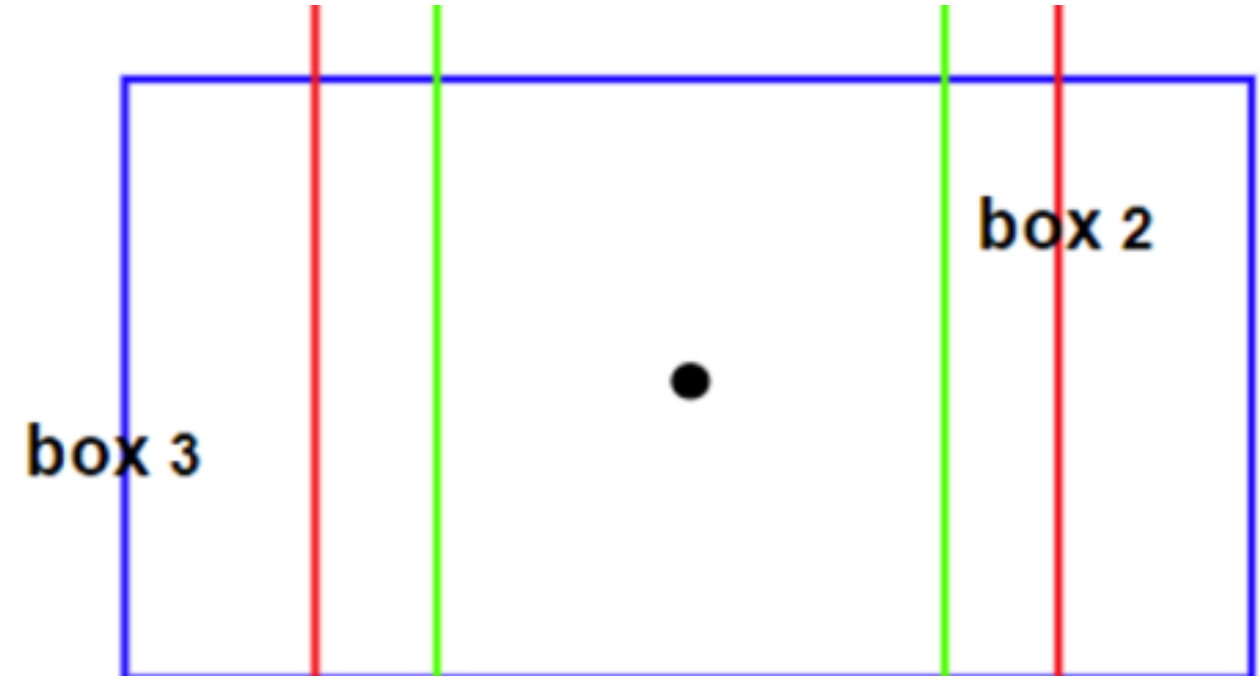



In about ai by Edgar Bermudez

## Understanding Embedding Models in the Context of Large Language Models

Large Language Models (LLMs) like GPT, BERT, and similar architectures have revolutionized the field of natural language processing (NLP)...

Jan 28 1




Wilasinee Paewboontra

## Enhance Object Detection Performance Through Anchor Box Optimization

The key configuration that significantly enhances object detection model performance is optimizing the size of anchor boxes. The anchor...

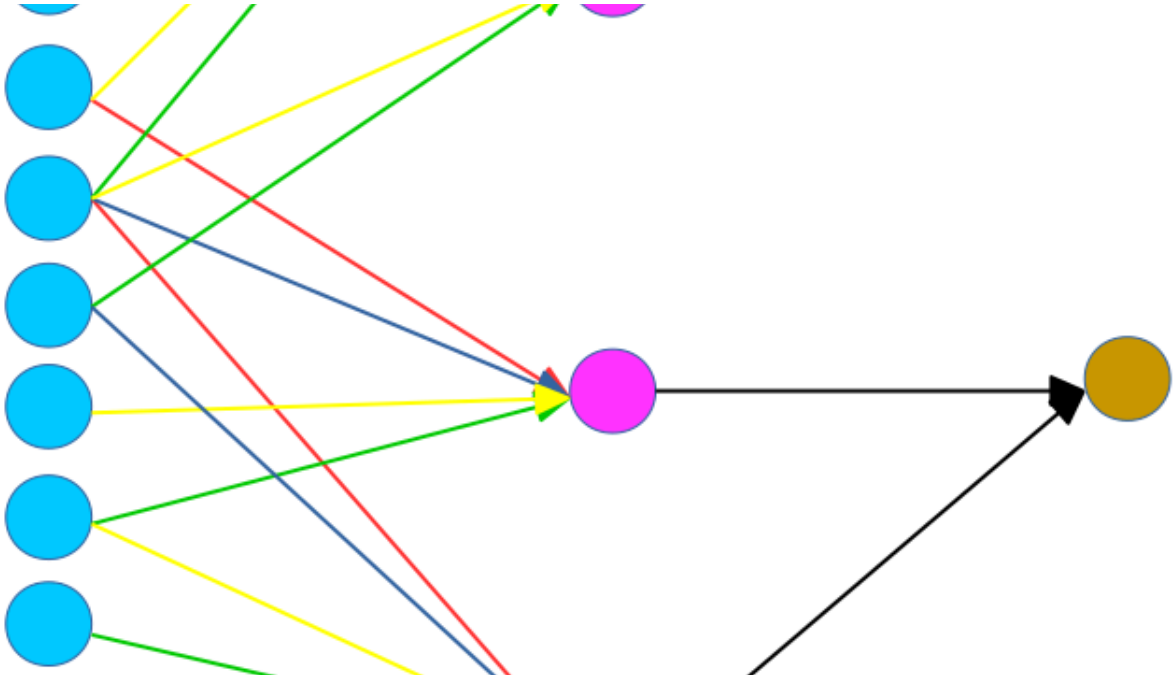
Dec 29, 2024 13


Sajid Khan

## Beginner's Guide to Video-Transformer (ViT) Model for Video Analytics

Building a Simple Video Transformer for Action Recognition

Nov 6, 2024

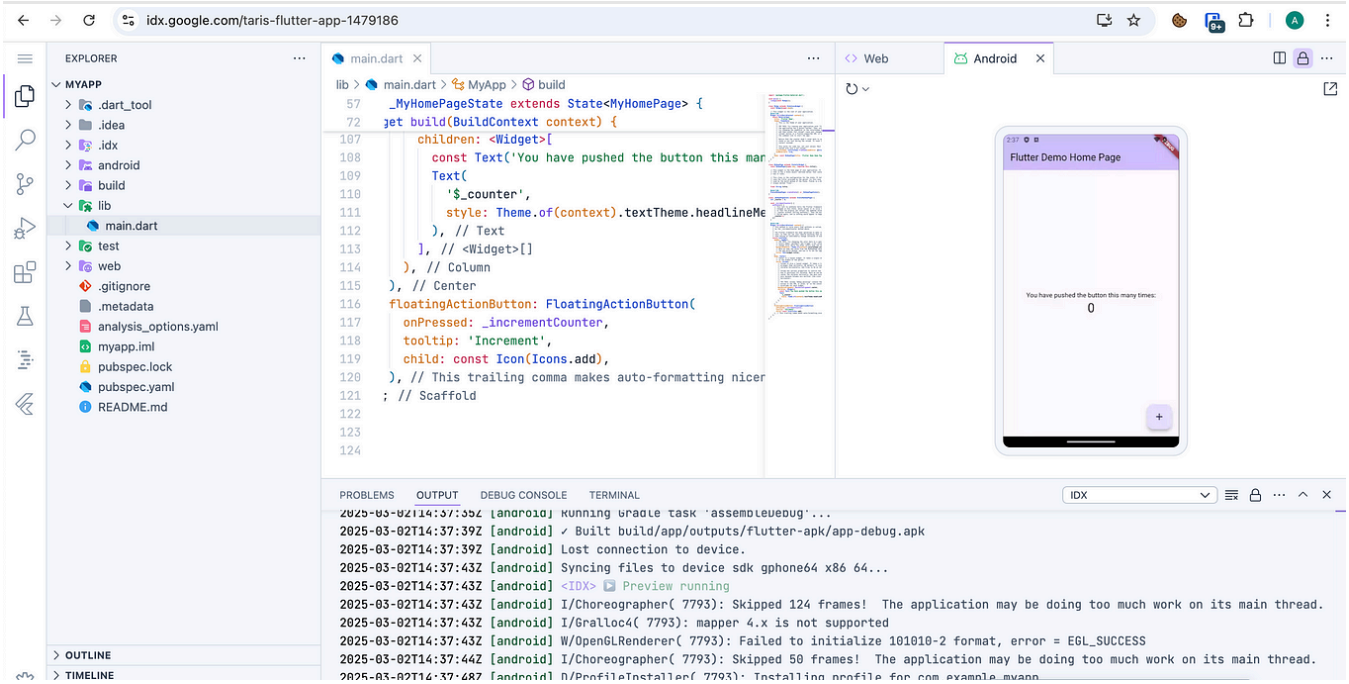


Jo Wang

# CNN Basics: Convolutional Layers and Pooling Layer | How to calculate parameters

Key Ingredient 1: Convolutional Layers

Oct 29, 2024 31




In Coding Beauty by Tari Ibaba

# This new IDE from Google is an absolute game changer

This new IDE from Google is seriously revolutionary.

Mar 11 4.9K 282

See more recommendations